

ROADMAP PENGEMBANGAN

FONDASI PERANGKAT LUNAK LAUNCHER

Dari simulasi desktop menuju kesiapan embedded dan hardware-in-the-loop

Dokumen roadmap teknis yang dapat dibaca lintas disiplin

Versi 1.2 | 01 Juli 2026

RUANG LINGKUP

Dokumen ini hanya membahas fondasi perangkat lunak: simulasi, flight software, navigation, verification, dan kesiapan integrasi hardware. Pendirian perusahaan, pendanaan, perizinan, desain mesin, struktur kendaraan, test stand, serta operasi peluncuran tidak termasuk dalam ruang lingkup.

Informasi Dokumen

Elemen	Keterangan
Tujuan	Menjadi acuan kerja bertahap untuk membangun fondasi perangkat lunak launcher dari nol tanpa melakukan reinventing the wheel.
Pembaca utama	Founder teknis, software engineer, embedded engineer, aerospace engineer, calon kolaborator, dan pihak nonteknis yang perlu memahami arah pengembangan.
Target awal	Sebuah virtual sounding rocket satu tahap yang dapat disimulasikan, dikendalikan oleh flight software virtual, diuji melalui berbagai skenario, dan dipersiapkan untuk integrasi hardware.
Horizon waktu	Sekitar 12–18 bulan untuk satu founder atau tim sangat kecil. Beberapa fase dapat dipercepat ketika tim tumbuh dan pekerjaan dijalankan paralel.
Batas akhir dokumen	Embedded readiness dan persiapan hardware-in-the-loop. Pekerjaan fisik dimulai setelah gerbang kesiapan tersebut terpenuhi.

Daftar Isi

1. Ringkasan Eksekutif
2. Prinsip Dasar Pengembangan
3. Target Produk Perangkat Lunak Versi Awal
4. Arsitektur Platform
5. Roadmap Fase 0–10
6. Workstream Pendukung
7. Strategi Build vs. Use
8. Timeline 12–18 Bulan
9. Kapan Mulai Mencari Tim
10. Milestone dan Ukuran Keberhasilan
11. Risiko dan Pola Kerja yang Harus Dihindari
12. Gerbang Menuju Hardware
13. Daftar Deliverable
14. Glosarium
15. Referensi Teknis

1. Ringkasan Eksekutif

Fondasi sebuah program launcher tidak harus dimulai dari mesin roket atau kendaraan fisik. Untuk founder dengan latar belakang software, titik awal yang paling realistis adalah membangun lingkungan simulasi dan flight software yang dapat diuji secara berulang, terukur, dan dipindahkan ke hardware pada tahap berikutnya.

Roadmap ini menempatkan perangkat lunak sebagai sarana untuk mengurangi risiko sebelum biaya pengembangan meningkat. Sistem dimulai dari model penerbangan virtual, kemudian ditambah sensor virtual, flight state machine, navigation, fault management, pengujian software-in-the-loop, Monte Carlo, guidance and control, dan akhirnya kesiapan embedded. Setiap fase menghasilkan bukti engineering yang dapat diperiksa, bukan sekadar demonstrasi visual.

Hasil yang ingin dicapai

Pada akhir roadmap, proyek memiliki platform software yang mampu menjalankan penerbangan virtual secara deterministik, menguji flight software terhadap sensor yang tidak sempurna, menyuntikkan kegagalan, merekam dan memutar ulang data, serta memindahkan flight core ke target embedded tanpa menulis ulang logika misi.

Apa yang belum dihasilkan

- Belum ada mesin roket, airframe, tangki, launch pad, atau kendaraan fisik.
- Belum ada desain propulsi, CFD mesin, injector, chamber, nozzle, atau sistem bahan bakar.
- Belum ada perangkat keras flight computer, sensor nyata, aktuator, atau test stand.
- Belum mencakup struktur perusahaan, pendanaan, perizinan, regulasi, dan operasi peluncuran.

Mengapa fondasi ini tetap bernilai

- Menjadi tempat aman untuk mengembangkan dan menguji algoritma sebelum hardware tersedia.
- Membentuk arsitektur reusable untuk sounding rocket, test stand, subscale vehicle, dan launcher yang lebih besar.
- Menghasilkan evidence: requirement, scenario, log, test result, dan regression suite yang dapat dinilai oleh engineer lain.
- Menjadi proof of engineering yang lebih kuat untuk menarik calon co-founder dan kolaborator teknis.

2. Prinsip Dasar Pengembangan

Prinsip	Makna Praktis
Mulai dari misi kecil yang lengkap	Lebih baik menyelesaikan virtual sounding rocket sederhana secara end-to-end daripada mengembangkan potongan software orbital yang tidak pernah terintegrasi.

Prinsip	Makna Praktis
Gunakan software yang sudah matang	Numerical solver, atmospheric model, RTOS, build system, dan framework umum tidak perlu dibuat ulang. Tim hanya membangun bagian yang menjadi pengetahuan dan keunggulan teknisnya.
Pisahkan ground truth dari sensor	Flight software tidak boleh membaca kondisi ideal simulator. Ia hanya boleh menerima data sensor virtual yang memiliki noise, delay, bias, dropout, dan keterbatasan nyata.
Requirement lebih dulu daripada fitur	Setiap modul harus memiliki alasan, input, output, batas, dan test yang jelas. Fitur tanpa requirement hanya menambah area kegagalan.
Deterministik dan dapat direproduksi	Konfigurasi, random seed, versi software, dan hasil pengujian harus dapat diulang oleh engineer lain.
Testing adalah produk utama	Nilai platform bukan hanya penerbangan yang berhasil, tetapi kemampuan membuktikan perilaku sistem pada kondisi nominal dan gagal.
Flight core harus portabel	Logika misi, navigation, dan fault response tidak boleh terikat pada Windows, Linux, sensor tertentu, atau simulator tertentu.
Kompleksitas ditambahkan bertahap	Active control, multi-stage, orbital guidance, dan high-fidelity model hanya ditambahkan setelah baseline sederhana stabil dan tervalidasi.

Definisi "selesai"

Sebuah fase tidak dianggap selesai hanya karena program dapat dijalankan. Fase selesai ketika hasilnya dapat diulang, batas keberhasilannya jelas, kegagalannya dapat didiagnosis, dan terdapat pengujian otomatis yang mencegah bug lama muncul kembali.

3. Target Produk Perangkat Lunak Versi Awal

Target awal adalah virtual sounding rocket satu tahap. Kendaraan ini sengaja dibuat sederhana agar seluruh rantai engineering dapat diselesaikan, mulai dari dinamika penerbangan hingga recovery. Tujuannya bukan mencapai performa tertinggi, melainkan membangun platform yang benar.

Area	Target Versi Awal	Belum Diperlukan
Kendaraan	Satu tahap, thrust curve generik, massa berubah, stabilitas aerodinamis pasif	Multi-stage dan reusable vehicle
Penerbangan	Peluncuran, powered ascent, coast, apogee, descent, landing	Orbit insertion dan reentry
Sensor	IMU, barometer, dan GNSS virtual	Sensor suite berlebih dan model industri beresolusi tinggi
Aktuator	Perintah recovery dan actuator emulator sederhana	Thrust vector control fisik
Flight software	State machine, event detection, estimation dasar, fault handling, logging	Autonomi penuh dan mission optimization
Pengujian	Nominal, sensor failure, komunikasi terganggu, restart, recovery failure	Qualification test hardware

Flight state yang harus didukung

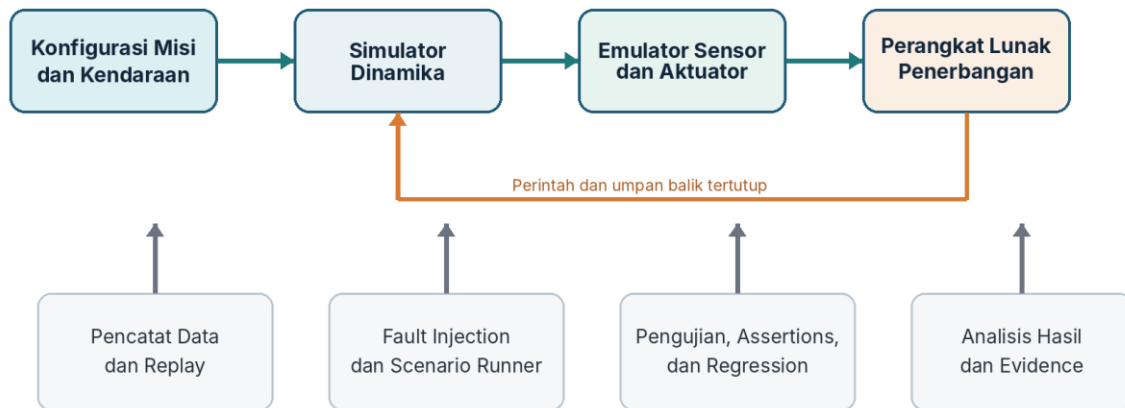
State	Tujuan
Boot	Perangkat lunak mulai, memeriksa kondisi dasar, dan menyiapkan layanan inti.
Safe	Sistem tidak diizinkan menjalankan tindakan berisiko.
Armed	Sistem siap mendeteksi peluncuran, tetapi masih memiliki inhibit yang jelas.
Launch Detected	Kendaraan dinilai benar-benar mulai bergerak berdasarkan beberapa kondisi.
Powered Ascent	Motor aktif dan kendaraan sedang mengalami fase dorong.
Coast	Motor selesai, kendaraan masih naik karena momentum.
Apogee	Titik tertinggi terdeteksi dengan guard dan validasi.
Descent	Kendaraan turun dan recovery system dikelola.
Landed	Kondisi mendarat terdeteksi secara stabil.

State	Tujuan
Fault / Abort	Sistem berpindah ke respons aman ketika kondisi penting tidak terpenuhi.

4. Arsitektur Platform

Platform dibagi menjadi beberapa komponen yang dapat berjalan terpisah. Pemisahan ini penting agar simulator, flight software, dan hardware nantinya dapat diganti tanpa menulis ulang seluruh sistem.

Arsitektur Fondasi Perangkat Lunak



Prinsip utama: ground truth simulator, data sensor, dan estimasi flight software harus tetap terpisah.

Gambar 1. Arsitektur tingkat tinggi fondasi perangkat lunak launcher.

Komponen utama

Komponen	Tanggung Jawab
Konfigurasi misi dan kendaraan	Menyimpan parameter kendaraan, lingkungan, target misi, batas keselamatan, serta scenario yang akan dijalankan.
Dynamics simulator	Menghitung kondisi kendaraan yang sebenarnya: posisi, kecepatan, percepatan, attitude, massa, aerodinamika, dan fase penerbangan.
Sensor emulator	Mengubah ground truth menjadi pembacaan sensor yang realistis, termasuk noise, bias, delay, quantization, saturation, dan dropout.

Komponen	Tanggung Jawab
Flight software	Menjalankan state machine, estimasi, event detection, fault response, command handling, telemetry, dan data recording.
Actuator emulator	Menerima perintah flight software dan menerapkan dampaknya kembali ke simulator.
Test controller	Menjalankan scenario, fault injection, assertions, timing control, dan pengumpulan hasil.
Replay dan evidence	Memutar ulang data, membandingkan versi algoritma, dan menyimpan bukti bahwa requirement telah diuji.

Tiga data yang tidak boleh dicampur

Ground truth adalah kondisi ideal dari simulator. Sensor reading adalah kondisi yang dilihat sensor. Estimated state adalah kondisi yang dihitung flight software. Kesalahan arsitektur paling berbahaya adalah membiarkan flight software menggunakan ground truth secara langsung.

5. Roadmap Fase 0–10

Setiap fase berikut memiliki tujuan, pekerjaan utama, deliverable, dan gerbang kelulusan. Urutan ini sengaja dibuat agar kompleksitas tumbuh bersama kemampuan verifikasi.

FASE 0	Definisi Misi dan Aturan Engineering <i>Menetapkan apa yang dibangun, apa yang tidak dibangun, dan bagaimana keberhasilan diukur.</i>
---------------	---

Fase ini mencegah tim menulis banyak software tanpa tujuan sistem yang jelas. Seluruh keputusan berikutnya harus dapat ditelusuri kembali ke misi dan requirement.

Pekerjaan utama

- Menetapkan misi virtual sounding rocket dan tanggung jawab flight software.
- Menyusun flight state, transition guard, dan kondisi abort.
- Menentukan satuan, coordinate frame, representasi waktu, dan konvensi data.
- Menyusun katalog scenario nominal dan kegagalan.
- Mendefinisikan batas ruang lingkup agar orbital mission, propulsion, dan hardware tidak masuk terlalu dini.

Elemen	Kriteria
Deliverable	Mission definition, software requirement awal, state diagram, interface overview, coordinate convention, dan scenario catalog.
Gerbang kelulusan	Semua pihak dapat menjelaskan misi, state kendaraan, input-output software, serta kondisi keberhasilan tanpa interpretasi yang bertentangan.

FASE 1**Baseline Dynamics Simulation**

Menghasilkan penerbangan virtual yang deterministik dan dapat divalidasi.

Gunakan framework simulasi yang sudah tersedia untuk dinamika atmosfer dan trajectory dasar. RocketPy dapat digunakan sebagai baseline karena menyediakan simulasi enam derajat kebebasan, efek perubahan massa, dan descent dengan parachute.[1]

Pekerjaan utama

- Membuat konfigurasi kendaraan, motor generik, lingkungan, dan misi.
- Menjalankan penerbangan nominal dari rail departure hingga landing.
- Menyimpan ground truth dengan format data dan metadata yang konsisten.
- Memvalidasi simulator menggunakan kasus sederhana yang dapat dihitung secara independen.
- Membuat golden scenario yang hasilnya dipakai sebagai baseline regression.

Elemen	Kriteria
Deliverable	Baseline simulator, vehicle configuration schema, nominal flight, ground-truth log, dan validation report.
Gerbang kelulusan	Konfigurasi dan seed yang sama menghasilkan output yang sama; kasus validasi sederhana sesuai ekspektasi; hasil dapat diputar ulang.

FASE 2**Emulasi Sensor dan Aktuator**

Membuat flight software menghadapi data yang tidak sempurna seperti pada sistem nyata.

Flight software hanya boleh melihat data sensor virtual. Sensor model tidak harus sempurna pada tahap awal, tetapi harus memiliki keterbatasan yang realistis dan dapat dikonfigurasi.

Sensor versi awal

Elemen	Model Minimum
IMU	Percepatan, angular rate, temperatur, noise, bias, drift, saturation, jitter, dan dropped sample.
Barometer	Pressure, altitude turunan, delay, noise, spike, slow response, dan frozen value.
GNSS	Position, altitude, ground speed, fix status, update rate rendah, delay, noise, dan complete dropout.
Actuator emulator	Menerima perintah recovery atau aktuator virtual dan mengubah kondisi simulator sesuai respons dan delay yang ditentukan.

Elemen	Kriteria
Deliverable	Sensor model library, actuator model, failure configuration, timestamp policy, dan sensor stream recorder.
Gerbang kelulusan	Flight software tidak memiliki akses ke ground truth; setiap fault dapat diaktifkan oleh scenario; output sensor dapat direproduksi.

FASE 3

Flight Software Core

Membangun logika penerbangan yang portabel dan tidak terikat pada hardware tertentu.

Flight core adalah bagian yang benar-benar menjadi aset engineering. Simulasi dapat menggunakan Python, tetapi flight core disarankan menggunakan C++ atau C agar mudah dipindahkan ke sistem embedded, memiliki kontrol memori yang jelas, dan dapat dijalankan secara deterministik.

Modul minimum

- Platform abstraction untuk clock, storage, communication, dan sensor access.
- State machine dan mission event handling.
- Estimation interface untuk altitude, velocity, dan attitude.
- Fault manager, inhibit, watchdog, dan safe-state response.
- Command handling, telemetry, dan data recording.
- Hardware abstraction agar simulator, replay, dan sensor nyata menggunakan interface yang sama.

Elemen	Kriteria
Deliverable	Portable flight core, interface contract, host platform adapter, dan unit test suite.
Gerbang kelulusan	Flight core dapat berjalan sebagai proses biasa di desktop, tidak membaca hardware secara langsung, dan tidak bergantung pada simulator tertentu.

FASE 4

State Machine dan Event Detection

Memastikan perubahan fase penerbangan terjadi berdasarkan bukti yang cukup dan urutan yang valid.

State transition penting tidak boleh dipicu oleh satu sensor sample. Setiap event harus menggunakan kombinasi kondisi, persistence time, inhibit, dan sanity check.

Event	Pendekatan
Launch detection	Kombinasi acceleration threshold, persistence, armed state, dan health sensor.
Motor burnout	Perubahan percepatan setelah minimum powered-flight duration dan validasi state sebelumnya.
Apogee detection	Perubahan estimated vertical velocity, minimum altitude guard, dan persistence beberapa sample.
Landing detection	Altitude dan velocity stabil, acceleration rendah, serta durasi stabil minimum.
Invalid transition	Transisi yang tidak sesuai urutan ditolak, dicatat, dan dapat memicu fault response.

Elemen	Kriteria
Deliverable	State machine lengkap, event detector, transition assertions, dan nominal mission test.
Gerbang kelulusan	Noise dan spike tidak menyebabkan false event; urutan state selalu valid; restart membawa sistem ke kondisi aman.

FASE 5**Software-in-the-Loop (SIL)**

Menguji flight software sebagai sistem terpisah, bukan fungsi internal simulator.

Simulator, flight software, dan test controller dijalankan sebagai proses terpisah. Struktur ini mendekati integrasi nyata dan memaksa tim mendefinisikan protocol, timing, timeout, serta perilaku ketika data terlambat atau hilang.

Kemampuan wajib

- Mode faster-than-real-time untuk regression dan Monte Carlo.
- Mode real-time untuk timing, latency, dan persiapan hardware-in-the-loop.
- Replay mode untuk menjalankan flight log lama pada versi software baru.
- Simulasi packet loss, corruption, delay, duplicate, dan out-of-order data.
- Pemisahan simulation clock dari wall clock.

Elemen	Kriteria
Deliverable	SIL runner, inter-process interface, real-time mode, replay pipeline, dan communication fault scenarios.
Gerbang kelulusan	Flight software tetap berfungsi ketika simulator diganti dengan replay source; hasil scenario otomatis dapat dinilai lulus atau gagal.

FASE 6**Navigation dan State Estimation**

Mengubah data sensor menjadi estimasi posisi, kecepatan, dan attitude beserta tingkat kepercayaannya.

Estimasi dibangun bertahap. Mulai dari vertical estimator sederhana yang dapat diukur, kemudian lanjut ke attitude estimator dan sensor health management. Model rumit tidak selalu lebih baik bila belum dapat divalidasi.

Urutan pengembangan

1. Vertical estimator menggunakan barometer, acceleration, dan GNSS altitude.
2. Pengukuran error terhadap ground truth: altitude error, velocity error, convergence time, dan event detection delay.
3. Attitude estimator menggunakan gyroscope, accelerometer, dan sensor tambahan bila dibutuhkan.
4. Sensor health melalui range check, stale data detection, rate-of-change check, dan cross-sensor disagreement.
5. Fallback mode ketika satu sensor hilang atau dianggap tidak sehat.

Elemen	Kriteria
Deliverable	Vertical estimator, attitude estimator baseline, sensor health monitor, uncertainty output, dan performance report.
Gerbang kelulusan	Error berada dalam batas requirement; kehilangan satu sensor tidak langsung menghentikan sistem; output selalu memiliki status health.

FASE 7

Fault Management

Menentukan bagaimana sistem mendeteksi, mengisolasi, dan merespons kondisi gagal.

Fault injection harus menjadi fitur utama platform. Tujuannya bukan membuat software yang tidak pernah gagal, tetapi membuat kegagalan terdeteksi, dapat dijelaskan, dan direspons secara aman.

Kelas Fault	Contoh	Respons yang Mungkin
Transient	Satu sensor sample hilang	Ignore terkontrol atau retry
Persistent	Barometer frozen	Isolasi sensor dan pindah sumber estimasi
Degraded	GNSS tidak tersedia	Lanjut dengan uncertainty lebih besar
Critical	Scheduler overrun berulang	Masuk safe state atau abort
Mission-ending	Recovery command tidak menghasilkan respons	Catat failure dan jalankan mitigasi yang tersedia

Elemen	Kriteria
Deliverable	Fault catalog, detection rule, response matrix, fault injection hooks, dan degraded-mode tests.
Gerbang kelulusan	Setiap fault penting memiliki detection, response, log, dan test; tidak ada error penting yang hanya dicetak lalu diabaikan.

FASE 8**Verification, Monte Carlo, dan Regression**

Membuktikan bahwa sistem bekerja pada rentang variasi, bukan hanya pada satu demo yang ideal.

Monte Carlo digunakan untuk memvariasikan massa, performa motor, aerodinamika, angin, sensor, delay, dan parameter lain. RocketPy menyediakan dukungan analisis dispersi Monte Carlo yang dapat dipakai sebagai fondasi.[1]

Lapisan pengujian

6. Unit test untuk fungsi dan class kecil.
7. Component test untuk satu modul dengan dependency terkontrol.
8. Integration test untuk aliran data antar-modul.
9. Software-in-the-loop untuk satu misi lengkap.
10. Fault injection untuk kegagalan terencana.
11. Monte Carlo untuk variasi parameter dan lingkungan.
12. Regression test untuk memastikan bug lama tidak kembali.

Elemen	Kriteria
Deliverable	Automated test pipeline, Monte Carlo runner, acceptance criteria, coverage report, result summary, dan traceability requirement-to-test.
Gerbang kelulusan	Setiap requirement memiliki test; setiap run menyimpan konfigurasi, seed, software version, dan hasil; kegagalan dapat direproduksi.

FASE 9**Guidance dan Control**

Menambahkan kemampuan mengarahkan kendaraan secara bertahap setelah estimator dan verification stabil.

Untuk demonstrator awal, stabilitas pasif sudah cukup. Active guidance and control baru dimulai setelah estimasi dan SIL matang agar controller tidak dikembangkan di atas data yang salah.

Urutan pengembangan

13. Single-axis control pada model sederhana.
14. Pitch dan yaw control dengan actuator limit dan delay.
15. Full attitude control dengan saturation, deadband, dan failed-actuator scenario.
16. Vertical attitude hold dan predefined pitch profile.
17. Angle-of-attack dan angular-rate limit.
18. Targeted apogee atau trajectory shaping setelah baseline stabil.

	Belum masuk orbital guidance Orbit insertion, multi-stage sequencing, trajectory optimization, dan mission design orbital membutuhkan roadmap lanjutan. NASA GMAT relevan ketika pekerjaan sudah beralih ke desain misi dan navigasi orbit.[5]
--	--

Elemen	Kriteria
Deliverable	Control sandbox, controller baseline, actuator model, guidance profile, stability report, dan closed-loop Monte Carlo test.
Gerbang kelulusan	Controller stabil pada rentang scenario yang ditentukan, menghormati actuator limit, dan masuk ke respons aman saat input atau actuator gagal.

FASE 10	Embedded Readiness dan Persiapan HIL <i>Membuktikan bahwa flight core siap dipindahkan dari desktop ke target embedded tanpa perubahan arsitektur besar.</i>
----------------	--

Pada fase ini hardware belum wajib tersedia, tetapi software harus disiapkan agar integrasi hardware tidak memaksa penulisan ulang. RTOS tidak perlu dibuat sendiri. Zephyr menyediakan kernel, driver ecosystem, device model, build system, dan hardware abstraction untuk berbagai target embedded.[2]

Syarat flight core

- Tidak melakukan akses OS atau hardware secara langsung.
- Tidak melakukan dynamic allocation pada jalur misi kritis setelah initialization.
- Menggunakan bounded buffer dan waktu eksekusi yang dapat diukur.
- Memiliki explicit time input dan tidak bergantung pada clock global tersembunyi.
- Dapat dikompilasi untuk host dan target embedded.
- Memiliki platform adapter untuk simulator, replay, desktop, dan RTOS.

F Prime dapat dievaluasi saat sistem mulai memiliki banyak komponen embedded, command, event, telemetry, dan deployment target. Framework ini bersifat component-driven dan dikembangkan untuk spaceflight serta embedded application.[3] cFS juga menyediakan arsitektur reusable dan platform-independent, tetapi lebih tepat dipertimbangkan ketika skala serta kebutuhan organisasi sudah cukup besar.[4]

Elemen	Kriteria
Deliverable	Cross-build pipeline, embedded platform adapter, timing budget, memory budget, scheduler model, HIL interface specification, dan board selection criteria.

Elemen	Kriteria
Gerbang kelulusan	Flight core dapat dibangun untuk target embedded pilihan; real-time scenario memenuhi timing budget; semua sensor dan actuator menggunakan interface yang siap dipetakan ke hardware.

6. Workstream Pendukung

Beberapa pekerjaan berjalan sepanjang seluruh fase. Tanpa workstream ini, proyek akan memiliki banyak source code tetapi sedikit pengetahuan yang dapat dipercaya.

Workstream	Hasil yang Diharapkan
Requirements dan traceability	Setiap requirement diberi identitas, rationale, owner, dan test yang membuktikannya.
Configuration management	Vehicle, environment, mission, fault, dan test configuration disimpan terpisah serta diberi versi.
Data governance	Setiap run menyimpan source revision, configuration hash, seed, timestamp, dan tool version.
Review engineering	Perubahan state machine, safety logic, estimator, dan interface kritis melalui design review dan code review.
Documentation	Arsitektur, interface, asumsi, keputusan, failure, dan limitation ditulis saat pekerjaan berlangsung.
Reproducible environment	Dependency dan build environment dapat dibuat ulang melalui lockfile, container, atau dev environment yang terdokumentasi.
Security dasar	Input validation, command authorization model, integrity check, dan audit log dipersiapkan sebelum ground system berkembang.
Knowledge base	Bug, eksperimen, hasil gagal, dan keputusan teknis disimpan agar tim baru tidak mengulangi kesalahan yang sama.

7. Strategi Build vs. Use

Tidak semua bagian layak dibangun sendiri. Perangkat lunak umum dipakai sebagai fondasi; tim berfokus pada model kendaraan, algoritma, mission logic, fault response, dan evidence yang menjadi pengetahuan khusus proyek.

Area	Keputusan	Rekomendasi Awal	Alasan
Atmospheric trajectory	Gunakan	RocketPy	Menghindari penulisan solver dan model dasar dari nol.[1]
Numerical computing	Gunakan	NumPy dan SciPy	Library matang untuk analisis dan estimasi.
Flight core	Bangun	C++ atau C portable	Menjadi IP, harus deterministik dan siap embedded.
Sensor model	Bangun	Library internal	Harus sesuai sensor, scenario, dan failure model proyek.
State machine	Bangun	Library internal	Mencerminkan mission logic dan safety behavior.
Navigation/GNC	Bangun bertahap	Algoritma internal di atas library numerik	Menjadi pengetahuan inti dan harus divalidasi terhadap vehicle model.
RTOS	Gunakan nanti	Zephyr	Tidak perlu membuat scheduler, driver ecosystem, dan hardware abstraction sendiri.[2]
Flight framework	Evaluasi nanti	F Prime atau cFS	Berguna setelah kompleksitas komponen dan organisasi meningkat.[3][4]
Spacecraft simulation	Gunakan nanti	Basilisk	Lebih cocok untuk spacecraft-centric dynamics dan GNC pada tahap orbital.[6]
Mission design orbital	Gunakan nanti	NASA GMAT	Relevan untuk trajectory design, optimization, dan navigation orbit.[5]
Build dan CI	Gunakan	CMake, test framework, static analysis, CI platform	Komponen umum tidak perlu dibuat sendiri.
Data format	Gunakan standar	Parquet/HDF5 dan metadata terstruktur	Memudahkan replay, analisis, dan reproduksi.

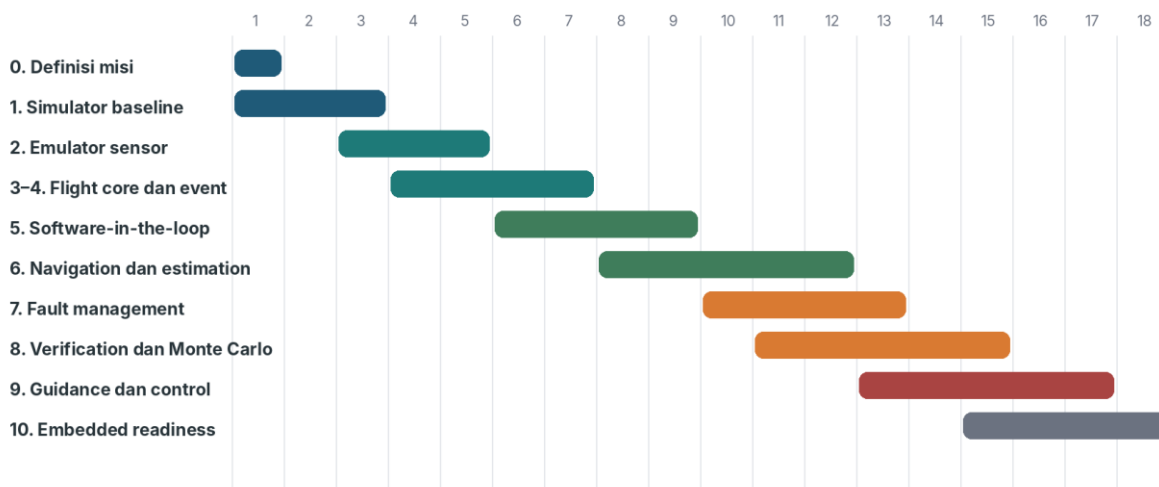
IP yang sesungguhnya

Nilai proyek bukan berasal dari membuat operating system, database, atau numerical solver baru. Nilainya berada pada vehicle-specific model, estimation, mission logic, guidance and control, fault response, scenario library, validation evidence, dan pengalaman operasional yang terakumulasi.

8. Timeline Implementasi 12–18 Bulan

Timeline berikut adalah baseline untuk satu founder atau tim sangat kecil. Durasi bukan janji kalender, melainkan urutan dependensi. Kualitas verification tidak boleh dikorbankan hanya agar fase tampak bergerak cepat.

Roadmap Implementasi 12–18 Bulan



Gambar 2. Baseline roadmap implementasi. Fase dapat overlap setelah interface stabil.

Periode	Fokus	Output Utama
Bulan 1–3	Mission definition dan baseline simulation	Requirement awal, state diagram, vehicle config, nominal simulation, ground-truth log.
Bulan 3–6	Sensor emulation dan flight core	Virtual sensors, portable flight core, state machine, event detection.
Bulan 6–9	SIL dan replay	Proses terpisah, real-time mode, communication fault, replay pipeline.
Bulan 8–13	Navigation dan fault management	Estimator, sensor health, degraded mode, fault response matrix.

Periode	Fokus	Output Utama
Bulan 11–15	Verification dan Monte Carlo	Scenario suite, automated evidence, regression, performance envelope.
Bulan 13–17	Guidance dan control	Closed-loop control baseline dan fault-tolerant behavior.
Bulan 15–18	Embedded readiness	Cross-build, timing and memory budget, HIL specification.

9. Kapan Mulai Mencari Tim

Mencari kolaborator terlalu awal menghasilkan percakapan tentang visi. Mencari mereka setelah proof of engineering menghasilkan percakapan tentang pekerjaan nyata. Waktu terbaik adalah ketika repository dapat dijalankan dan satu misi virtual sudah memiliki hasil yang dapat diperiksa.

Titik Kematangan	Orang yang Dicari	Alasan
Setelah Fase 1–2	Aerospace atau simulation engineer	Mereview vehicle model, asumsi aerodinamika, dan validity simulator.
Setelah Fase 2–4	Embedded engineer	Membantu hardware abstraction, timing discipline, sensor interface, dan portability.
Setelah Fase 5–6	GNC / control engineer	Mengembangkan estimator, navigation, guidance, dan controller yang dapat divalidasi.
Menjelang Fase 10	Electrical dan avionics engineer	Menentukan board, power, sensor, bus, redundancy, dan HIL architecture.
Setelah software gate tercapai	Mechanical, propulsion, test, dan manufacturing engineer	Memulai program fisik terpisah dengan fondasi software dan verification yang sudah tersedia.

Proof of engineering minimum sebelum rekrutmen aktif

- Repository rapi dan dapat dijalankan oleh orang lain.
- Nominal virtual flight selesai dari boot sampai landing.
- Sensor simulator dan state machine sudah bekerja.
- Setidaknya satu failure scenario menghasilkan respons yang benar.
- Dokumentasi requirement, architecture, dan roadmap tersedia.
- Hasil test otomatis dapat dibaca tanpa membuka dashboard.

10. Milestone dan Ukuran Keberhasilan

Keberhasilan tidak diukur dari jumlah fitur atau banyaknya baris kode. Gunakan milestone yang menunjukkan penurunan risiko dan peningkatan kemampuan verifikasi.

Milestone	Bukti Minimum	Risiko yang Berkurang
M1: Mission Baseline	Mission, state, scenario, dan requirement disepakati	Scope tidak terkendali
M2: Deterministic Flight	Nominal simulation dapat diulang dan tervalidasi	Model dasar tidak dipercaya
M3: Sensor Reality	Flight software hanya menerima sensor virtual	Ketergantungan pada data ideal
M4: Mission Logic	State dan event valid pada nominal dan noisy scenario	False event dan invalid transition
M5: Independent FSW	Simulator dan flight software berjalan terpisah	Arsitektur tidak siap integrasi
M6: Reliable Estimation	Error terukur dan degraded mode bekerja	Keputusan berdasarkan estimasi yang salah
M7: Fault Evidence	Fault catalog dan response diuji otomatis	Kegagalan tidak terdeteksi
M8: Statistical Confidence	Monte Carlo memenuhi acceptance criteria	Demo hanya berhasil pada satu kondisi
M9: Closed-loop Control	Controller stabil dan menghormati limit	Active control tidak terkendali
M10: Embedded Ready	Cross-build, timing, memory, dan HIL spec tersedia	Penulisan ulang saat pindah ke hardware

Contoh kategori metrik

Kategori	Metrik yang Dapat Digunakan
Kebenaran misi	Valid state transition, event detection success, false event count, recovery command timing.
Kualitas estimasi	Altitude error, velocity error, attitude error, convergence time, uncertainty consistency.
Keandalan	Crash rate, watchdog event, fault detection coverage, degraded-mode success.

Kategori	Metrik yang Dapat Digunakan
Performa waktu	Loop execution time, jitter, deadline miss, communication latency.
Reproducibility	Run yang dapat diulang, traceability lengkap, failure yang dapat direproduksi.
Testing	Requirement coverage, scenario coverage, regression stability, Monte Carlo pass rate.

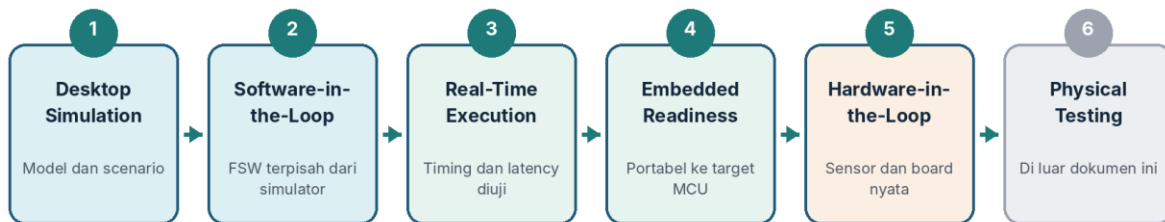
11. Risiko dan Pola Kerja yang Harus Dihindari

Risiko / Anti-Pattern	Dampak	Mitigasi
Membangun physics engine dari nol	Waktu habis pada solver umum, hasil belum tentu valid	Gunakan framework matang dan validasi vehicle-specific model.
Memulai dari UI dan dashboard	Tampilan berkembang lebih cepat daripada kemampuan sistem	Gunakan CLI, report, dan file output sampai core stabil.
Flight software membaca ground truth	Hasil tampak bagus tetapi tidak realistis	Paksa semua input melalui sensor abstraction.
Membuat satu aplikasi monolitik	Sulit mengganti simulator dengan replay atau hardware	Pisahkan proses dan kontrak interface.
Langsung memakai framework besar	Tim sibuk mempelajari framework sebelum memahami misi	Bangun portable core kecil, evaluasi F Prime/cFS saat kompleksitas nyata muncul.
Algoritma terlalu rumit terlalu awal	Sulit divalidasi dan diagnosis error tidak jelas	Mulai dari estimator dan controller sederhana dengan metrik yang terukur.
Tidak menyimpan seed dan konfigurasi	Bug tidak dapat direproduksi	Simpan metadata lengkap pada setiap run.
Testing dilakukan di akhir	Arsitektur sulit diuji dan bug menumpuk	Buat test harness sejak fase pertama.
AI anomaly detection terlalu dini	Model menutupi aturan engineering yang belum matang	Mulai dari threshold, residual, health rule, dan deterministic fault logic.
Masuk hardware sebelum gate	Biaya naik sementara software belum dapat dipercaya	Tahan pengadaan hardware sampai SIL, replay, dan fault test stabil.

12. Gerbang Menuju Hardware

Hardware mulai masuk setelah software menunjukkan bahwa ia dapat dipindahkan, diuji, dan didiagnosis. Ini bukan berarti seluruh algoritma sudah sempurna, tetapi fondasinya tidak lagi bergantung pada kondisi virtual tertentu.

Jalur Kematangan: Dari Laptop Menuju Hardware



Batas dokumen: selesai pada kesiapan embedded dan rencana HIL. Desain mesin, airframe, test stand, dan peluncuran fisik dibahas dalam roadmap terpisah.

Gambar 3. Jalur kematangan dan batas ruang lingkup dokumen.

Checklist gerbang

- Simulator dapat berjalan faster-than-real-time dan real-time.
- Flight software berjalan sebagai sistem terpisah dan tidak membaca ground truth.
- Sensor dan actuator memakai interface yang dapat diganti.
- Nominal mission, noise, dropout, restart, dan communication fault telah diuji.
- Replay system dapat menjalankan data lama pada software baru.
- Estimator memiliki error limit dan health status.
- Fault response terdefinisi dan terbukti melalui automated test.
- Monte Carlo memenuhi acceptance criteria awal.
- Flight core dapat dikompilasi untuk target embedded.
- Timing budget, memory budget, dan HIL interface specification tersedia.

Setelah gerbang ini

Program fisik dapat dimulai melalui pemilihan board, sensor nyata, power system, data acquisition, bench testing, dan hardware-in-the-loop. Pada titik tersebut diperlukan roadmap avionics, propulsion, structure, test, dan manufacturing yang terpisah.

13. Daftar Deliverable

Daftar ini dapat dipakai sebagai backlog tingkat program. Format dan nama file bebas, tetapi seluruh substansi harus tersedia dan terus diperbarui.

Kategori	Deliverable
Mission dan requirement	Mission definition, scope, flight state, software requirement, safety assumption, acceptance criteria.
Architecture	System context, component responsibility, interface contract, timing model, coordinate frame, data flow.
Simulation	Vehicle model, environment model, nominal scenario, validation case, ground-truth output, model limitation.
Sensor dan actuator	Sensor model, failure model, actuator response, timestamp policy, sampling policy.
Flight software	Portable core, state machine, event detection, estimator, fault manager, command, telemetry, recorder.
Testing	Unit test, integration test, SIL scenario, replay test, fault injection, Monte Carlo, regression suite.
Evidence	Test report, performance report, requirement traceability, failure report, known limitation.
Embedded readiness	Cross-build, platform adapter, RTOS evaluation, memory budget, timing budget, HIL specification.
Knowledge	Architecture decision record, engineering note, experiment log, glossary, onboarding guide.

14. Glosarium

Istilah	Definisi Ringkas
6-DOF	Six degrees of freedom: pergerakan translasi dan rotasi kendaraan dalam tiga sumbu.
Actuator	Komponen yang menjalankan perintah, misalnya servo, recovery mechanism, atau engine gimbal.
Apogee	Titik tertinggi lintasan sebelum kendaraan mulai turun.
Deterministik	Input, konfigurasi, dan seed yang sama menghasilkan output yang sama.
Digital twin	Model digital yang telah disinkronkan dan dikalibrasi menggunakan data dari aset fisik nyata. Sebelum hardware tersedia, istilah yang lebih tepat adalah simulation and verification platform.

Istilah	Definisi Ringkas
Embedded system	Komputer khusus dengan resource terbatas yang menjalankan fungsi tertentu pada perangkat.
Estimator	Algoritma yang memperkirakan kondisi kendaraan dari data sensor yang tidak sempurna.
Fault injection	Penyisipan kegagalan secara sengaja untuk menguji detection dan response.
Flight software	Perangkat lunak yang berjalan pada flight computer dan mengelola misi, state, estimation, command, fault, serta data.
GNC	Guidance, Navigation, and Control: menentukan tujuan, memperkirakan kondisi, dan menghasilkan tindakan kendali.
Ground truth	Kondisi sebenarnya menurut simulator, sebelum diubah menjadi pembacaan sensor.
Hardware-in-the-loop	Pengujian yang menghubungkan hardware nyata dengan simulator real-time.
Monte Carlo	Pengujian berulang dengan variasi parameter untuk memahami distribusi hasil dan batas performa.
Regression test	Test permanen yang memastikan perubahan baru tidak merusak perilaku yang sebelumnya benar.
RTOS	Real-time operating system yang dirancang untuk tugas dengan kebutuhan timing yang dapat diprediksi.
SIL	Software-in-the-loop: flight software asli diuji terhadap simulator melalui interface yang mendekati integrasi nyata.

15. Referensi Teknis

Referensi berikut adalah sumber resmi yang digunakan untuk menempatkan rekomendasi tools dalam roadmap. Pemilihan final tetap harus melalui proof of concept dan review kebutuhan proyek.

[1] RocketPy Documentation

Open-source Python library untuk simulasi trajectory roket 6-DOF, perubahan massa, parachute descent, dan analisis Monte Carlo. [Sumber resmi](#)

[2] Zephyr Project Documentation

RTOS open source dengan kernel, device model, driver ecosystem, build system, dan hardware abstraction.

[Sumber resmi](#)

[3] F Prime Documentation

Component-driven framework untuk embedded systems dan spaceflight software yang dikembangkan di NASA JPL. [Sumber resmi](#)

[4] NASA core Flight System

Reusable dan platform-independent embedded flight software framework. [Sumber resmi](#)

[5] NASA General Mission Analysis Tool

Open-source software untuk space mission design, optimization, dan navigation. [Sumber resmi](#)

[6] Basilisk Documentation

Open-source spacecraft-centric mission simulation framework untuk dynamics dan GNC. [Sumber resmi](#)

Kesimpulan

Langkah pertama bukan membuat kendaraan fisik dan bukan membangun seluruh software dari nol. Langkah pertama adalah membentuk misi kecil yang lengkap, menghubungkan simulator dengan flight software yang portabel, lalu membuktikan perilakunya melalui pengujian. Ketika platform telah mencapai embedded readiness, tim dapat masuk ke hardware dengan risiko yang jauh lebih terkendali.